

Franck-Hertz Experiment

Berkay Köktürk
Boğaziçi University • Physics Department

Abstract—The Franck–Hertz experiment provides direct evidence of quantized atomic energy levels through inelastic electron–atom collisions. In this work, the experiment was performed using a neon discharge tube with automated voltage control and current acquisition. Current–voltage characteristics were measured for multiple accelerating potentials, and Gaussian fits were applied to the observed current maxima to determine excitation energies and uncertainties. Weighted averaging across ten datasets yielded a first excitation energy of $E = 19.12 \pm 0.05$ eV, consistent with the expected neon excitation near 18.7 eV. These results confirm the quantized nature of electron excitation in neon using modern data-driven analysis on a classic quantum experiment.

I. INTRODUCTION

A. History

In 1911, James Franck and Gustav Hertz, two German physicists, first conceived an experiment to explore electron conduction in a gaseous medium. At that time, the idea of quantized atomic energy levels had not yet been fully developed. Their early assumption was that electrons would collide elastically with atoms until reaching a certain kinetic energy.

In 1914, after Niels Bohr had published his atomic model only one year earlier, Franck and Hertz performed their landmark experiment using mercury vapor. They observed periodic drops in the collected current as electrons were accelerated through the gas, demonstrating that electrons lose energy in discrete amounts. This corresponded to an excitation energy of approximately 4.9 eV, providing one of the earliest experimental confirmations of quantized atomic levels.

In 1915, Bohr highlighted that the results of the Franck–Hertz experiment were consistent with his model of discrete atomic states, further strengthening the foundation of quantum theory. Franck and Hertz were later awarded the Nobel Prize in 1926 “for their discovery of the laws governing the impact of an electron upon an atom.”[1]

B. Theory

The Franck–Hertz experiment provides a direct demonstration that atoms absorb energy in discrete quanta. In the original mercury setup, periodic drops in the collected current revealed the first inelastic excitation near 4.9 eV, confirming quantized atomic levels shortly after the Bohr model was introduced. Modern versions use noble gases such as neon, which offer more stable discharge and visible emission lines, making the excitation process easier to observe. Although the relevant excitation energies differ — with neon exhibiting a higher first effective excitation energy — the underlying physics remains unchanged: electrons transfer energy in discrete amounts during collisions with atoms.

Electron Acceleration and Inelastic Collisions

[2]

Electrons are thermionically emitted and accelerated by an applied voltage. As the accelerating potential increases, electrons gain kinetic energy until they reach the excitation energy of a neon atom. Below this threshold, collisions with neon atoms are predominantly elastic, and the collector current increases smoothly with the accelerating voltage. Once the electron energy matches the excitation energy of neon, inelastic collisions occur, and electrons transfer nearly all of their kinetic energy to the atomic system. As a consequence, fewer electrons reach the collector, and the measured current exhibits a distinct drop. When the accelerating voltage increases further, electrons again accumulate sufficient energy to excite another neon atom, resulting in periodic minima in the current–voltage curve.

These periodic minima directly reflect quantized atomic excitation, since the difference between successive minima corresponds to the effective excitation energy:

$$\Delta E \approx e \Delta V, \quad (1)$$

where ΔV is the voltage spacing between two minima.

Neon Excitation Structure (Fig.1)

Neon possesses two closely-spaced excited states at approximately 16.6 eV and 18.7 eV. Despite the existence of the lower 16.6 eV level, the 18.7 eV level dominates the inelastic scattering signal due to its larger excitation cross section and allowed transition characteristics.[3] Additionally, decay from the 16.6 eV state to the ground state is strongly suppressed by selection rules, making its contribution negligible in the observed current minima.

Thus, the first noticeable minimum in the Franck–Hertz curve corresponds to electrons exciting neon atoms to the 18.7 eV level.

Once in the excited state, neon atoms relax through intermediate levels, commonly transitioning from the 18.7 eV level to the 16.6 eV level and producing a photon with energy

$$\Delta E = 18.7 - 16.6 = 2.1 \text{ eV}. \quad (2)$$

This corresponds to a wavelength[4]

$$\lambda = \frac{hc}{E} \approx 590\text{--}620 \text{ nm}, \quad (3)$$

which lies in the orange region of the visible spectrum. This emission is responsible for the orange glow observed in the tube during the experiment.

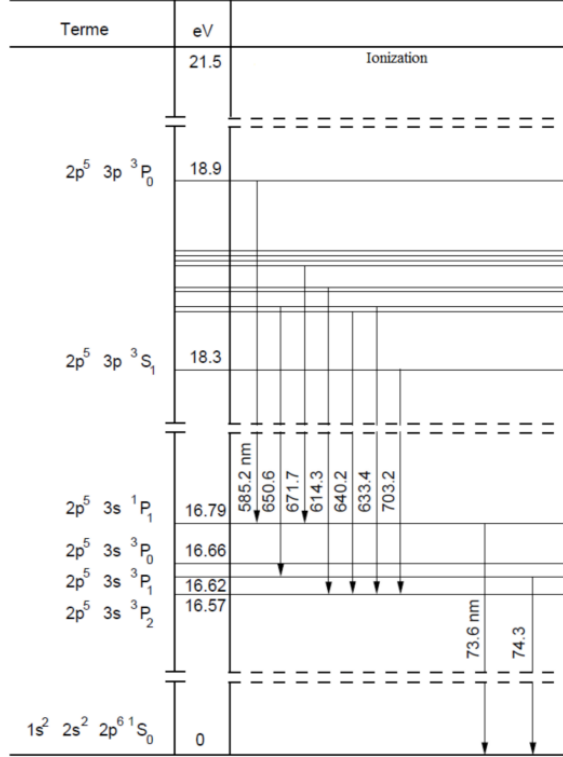


Fig. 1. Excitation levels of neon. The 18.7 eV state dominates the scattering probability, whereas the 16.6 eV state contributes weakly due to selection rules and lower cross section.

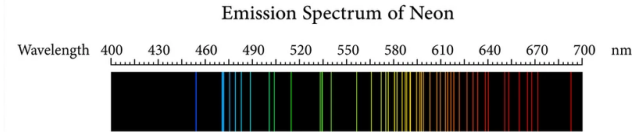


Fig. 2. Representative emission spectrum of neon, illustrating prominent orange emission near 600 nm.

Quantum Mechanical Origin of the Dominant 18.7 eV Transition in Neon

Although neon possesses two low-lying excited configurations near 16.6 eV and 18.7 eV above the ground state, only the latter produces a pronounced minimum in the Franck–Hertz curve. This behavior arises from the quantum structure of neon and the selection rules governing inelastic electron–atom collisions.

In multi-electron atoms such as neon, energy levels are labelled by the Russell–Saunders (LS) coupling scheme[5], where the total orbital and spin angular momenta satisfy

$$\vec{L} = \sum_i \vec{\ell}_i, \quad \vec{S} = \sum_i \vec{s}_i, \quad \vec{J} = \vec{L} + \vec{S}. \quad (4)$$

The fine structure splitting originates from the spin–orbit Hamiltonian

$$\hat{H}_{SO} = \xi(r) \vec{L} \cdot \vec{S}, \quad (5)$$

which lifts the degeneracy between states of identical L and S but different J .

The lowest excited configuration of neon ($2p^5 3s$) therefore splits into multiple J -resolved levels. The state at 16.6 eV corresponds to a metastable 3P_2 configuration, whereas the dominant 18.7 eV level arises from the optically allowed 1P_1 state. Electric dipole transitions obey the selection rules

$$\Delta S = 0, \quad \Delta L = \pm 1, \quad \Delta J = 0, \pm 1 \quad (J = 0 \not\rightarrow J = 0), \quad (6)$$

and the dipole transition amplitude is proportional to the matrix element

$$M_{if} \propto \langle \psi_f | \hat{\mathbf{d}} | \psi_i \rangle, \quad (7)$$

where $\hat{\mathbf{d}} = e \sum_i \vec{r}_i$ is the electric dipole operator. For the 16.6 eV triplet state, this matrix element vanishes (or is strongly suppressed) due to the spin selection rule $\Delta S = 0$, making the state optically forbidden and metastable. Consequently, its lifetime τ scales approximately as

$$\tau^{-1} \propto |M_{if}|^2, \quad (8)$$

implying τ in the range of μs to ms for the 16.6 eV state, in contrast to ns-scale decay for the 18.7 eV state.

During electron bombardment, the inelastic collision cross section is proportional to the same transition matrix element,

$$\sigma_{\text{inel}}(E) \propto |M_{if}|^2 f(E), \quad (9)$$

where $f(E)$ accounts for the energy distribution of incident electrons. Since $|M|^2$ for the metastable level is orders of magnitude smaller, excitation to 16.6 eV is both rare and does not measurably reduce the collector current.

By contrast, excitation to the 18.7 eV 1P_1 state is dipole-allowed, has a much larger inelastic collision cross section, and results in prompt radiative decay. Therefore, a significant fraction of electrons lose nearly all of their kinetic energy at this threshold, leading to the pronounced periodic minima in the measured current. Thus, the observed Franck–Hertz excitation energy in neon corresponds to the 18.7 eV level, despite the existence of a nearby lower metastable excited state.

Energy Broadening and Mean Free Path Effects

[6]

In practice, the minima in the I – V curve are not perfectly sharp. Finite gas pressure, space-charge effects near the cathode, and the spread in initial electron energies lead to broadening of the observed features. The mean free path plays an essential role: if the pressure is too low, electrons gain more energy before colliding, which may shift the effective excitation point; if the pressure is too high, electrons lose energy through frequent elastic collisions before gaining sufficient excitation energy.

Therefore, the observed voltage spacing reflects not only the intrinsic excitation energy but also the interplay between electron acceleration, collision probability, and energy distribution.

In summary, the neon Franck–Hertz experiment verifies discrete atomic excitation energies through periodic suppression of collector current. The extracted voltage spacing between minima directly yields the first effective excitation energy of neon, demonstrating the quantum nature of electron–atom interactions in a controlled and visually observable way.

II. METHOD

In this section, we will explain the procedures we followed during the experiment. We can see the electrical scheme of the Franck-Hertz tube in Fig.3

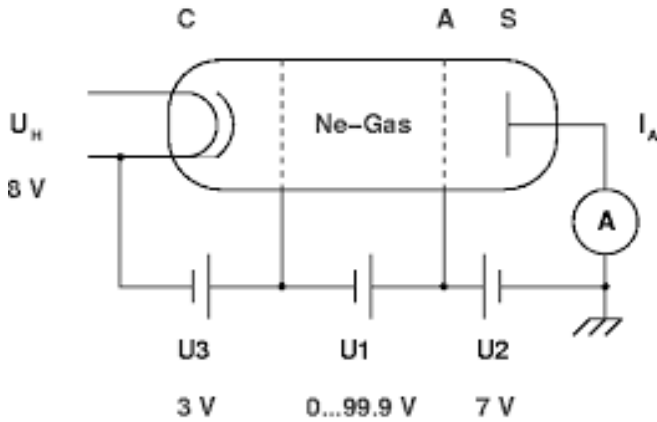


Fig. 3. Schematic diagram for electrical circuit of Franck-Hertz tube filled with Ne gas. As described in [7]

We conducted the experiment 10 times and collected the 10 different data sets. We tried to observe the relationship between voltage U_1 and current I for different values of voltages U_1 and U_3 . Here in this scheme, U_1 is the accelerating potential, which varies between 1V and 100V. The voltage labeled U_H is the potential applied to the filament to make it emit electrons. We kept fixed its value during the experiment. The voltages U_3 and U_2 are the initial accelerating potential and stopping potential, respectively, which can be arranged to values between 1 – 5V and 1 – 10V.

Step-by-step instructions for conducting the experiment are given below.

- 1) Turn on the power supply unit and set the function to "PC".
- 2) From the control panel in PC, set all the potentials to default values and take the first data set.
- 3) Take a total of 10 data sets by varying the values of U_2 and U_3 . **DO NOT** set them above the default values, which are 5V for U_3 and 10V for U_2 .
- 4) Make sure that on the graphs you have at least four distinct visible peaks. If not, then discard that data set and take a new one with different voltage values.
- 5) After collecting 10 different data sets, make sure that every instrument is turned off and the supply unit is unplugged.[8]

III. THE EXPERIMENTAL SETUP

We will make a list of the equipment that we used in the experiment. The Franck-Hertz experiment setup can be seen in Fig.4. We used a modern version of the setup with a Ne tube. In this setup, most of the work is done by the computer, and the control unit is fully automated. The work that needs to be done manually is almost none.

- 1) **Control Unit:** It provides and stabilizes all electrical parameters required for the experiment. It controls filament



Fig. 4. Experimental setup and components of Franck-Hertz experiment with Ne tube.

heating for thermionic emission, supplies accelerating and retarding (stopping) voltages, and measures the collector current. Key functions include:

- Supplying cathode heating current (which is created by U_H) for electron emission,
 - Applying the initial voltage U_3 to align the electrons emitted by the filament to the grid. This potential is related with the number of electrons in the beam i.e. beam density,
 - Applying the accelerating voltage U_1 to provide the kinetic energy to electrons passed the first grid.
 - Applying the retarding (stopping) voltage U_2 to filter low-energy electrons,
 - Measuring the collector current I_A ,
 - Providing overload protection and voltage/current stabilization.
- 2) **Ne Tube:** It contains low-pressure neon gas and multiple electrodes responsible for generating and accelerating the electron beam. Its internal structure includes:
 - A heated cathode for thermionic electron emission,
 - A first grid for electron acceleration,
 - A second grid where retardation and energy control occur,
 - An anode (collector) that records electrons that did not undergo inelastic collisions.
 - 3) **Computer:** Unfortunately, it is not seen in Fig.4 but it is the crucial part of the experiment. A computer is used to control voltage sweeps and acquire experimental data. It automates the measurement of the collector current as a function of accelerating voltage and displays the resulting $I-V$ curve in real time. Its functions include:
 - Performing automated and precise voltage sweeps,
 - Recording collector current I_A at each voltage step,
 - Filtering noise and averaging repeated measurements,
 - Exporting data for post-processing and analysis.

IV. THE DATA

In this part, normally, we need to share our data sets. However, each data set includes more than 800 data points, so this makes sharing whole data sets as a table impossible. Thus, we put a drive link in Appendix for you to acquire the data sets we use in the following analysis part.

V. THE ANALYSIS

In this section, we perform the analysis of the experiment with given data sets in the Appendix below. We first plot all the data points on a graph with their uncertainties. Apply a Gaussian fit on the top of each peak that is visible on the graphs. We record the mean and the uncertainty of the mean for each peak and calculate the quantity $\Delta E_i = \mu_{i+1} - \mu_i$ with appropriate error propagation. Then we calculate the weighted average of all ΔE_i 's for each graph. We create a weighted histogram. Finally, the weighted average of the histogram should give us the first excitation level of the Ne atom.

An important question is what the uncertainty of the given data sets is. We can see that the voltage values have constant step and it is a controlled parameter. So, the type-B uncertainty is more suitable for this variable. It is given by division of the interval range by the total step. i.e. if the steps are constant, then it corresponds to the half-step.

$$\sigma_V = \pm 0.05V \quad (10)$$

The other parameter we have is the current. The current is not the controlled parameter, yet it is observed. The type-A uncertainty is more suitable for this parameter. The type-A uncertainty is given by the standard deviation of the data sets. Since it goes up and down very fast in some intervals and is nearly constant in some others, we need to restrict our focus to specific intervals. However, we also need to consider the measurement precision of the device. So, we need to use the following formula for the current.

$$\sigma_{I,\text{total}} = \sqrt{\sigma_{I,A}^2 + \sigma_{I,B}^2} \quad (11)$$

We can measure this quantity, but it has a high chance to differ from one data set to another. To contain the uncertainty in the regions beyond the plateaus and get the same uncertainty for all data sets, we take its value as the step size.

$$\sigma_I = \pm 0.01\text{nA} \quad (12)$$

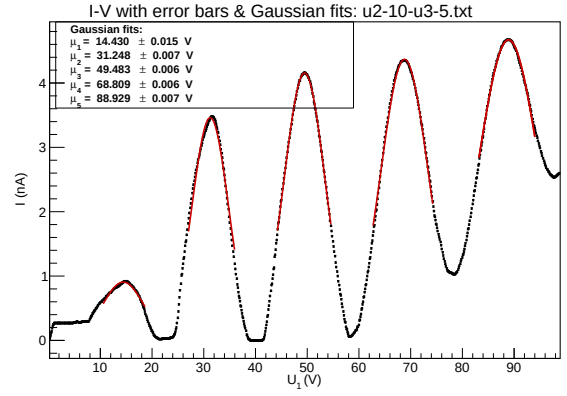


Fig. 5. Plot with error bars and Gaussian fits for the values of $U_2 = 10V$ and $U_3 = 5V$.

Energy Differences Between Peaks for Fig.5

Peak Difference	$\Delta E_i \pm \sigma_{\Delta E_i}$
ΔE_1	$16.817 \pm 0.016 \text{ eV}$
ΔE_2	$18.235 \pm 0.009 \text{ eV}$
ΔE_3	$19.326 \pm 0.009 \text{ eV}$
ΔE_4	$20.121 \pm 0.009 \text{ eV}$
Weighted Average	$18.997 \pm 0.005 \text{ eV}$

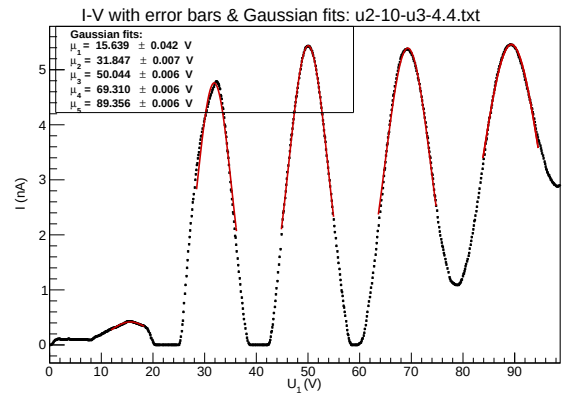


Fig. 6. Plot with error bars and Gaussian fits for the values of $U_2 = 10V$ and $U_3 = 4.4V$.

Energy Differences Between Peaks for Fig.6

Peak Difference	$\Delta E_i \pm \sigma_{\Delta E_i}$
ΔE_1	$16.208 \pm 0.042 \text{ eV}$
ΔE_2	$18.197 \pm 0.009 \text{ eV}$
ΔE_3	$19.266 \pm 0.008 \text{ eV}$
ΔE_4	$20.045 \pm 0.009 \text{ eV}$
Weighted Average	$19.136 \pm 0.005 \text{ eV}$

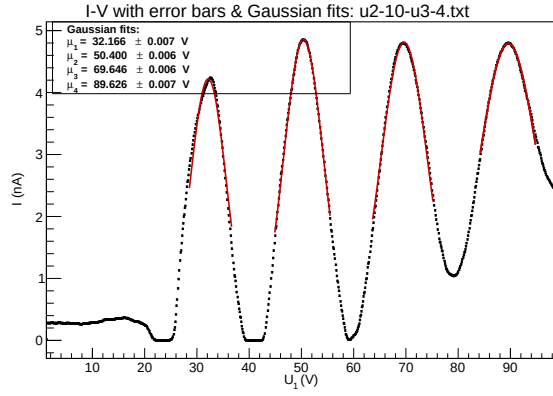


Fig. 7. Plot with error bars and Gaussian fits for the values of $U_2 = 10\text{V}$ and $U_3 = 4\text{V}$.

Energy Differences Between Peaks for Fig.7

Peak Difference	$\Delta E_i \pm \sigma_{\Delta E_i}$
ΔE_1	$18.234 \pm 0.009 \text{ eV}$
ΔE_2	$19.246 \pm 0.008 \text{ eV}$
ΔE_3	$19.980 \pm 0.009 \text{ eV}$
Weighted Average	$19.161 \pm 0.005 \text{ eV}$

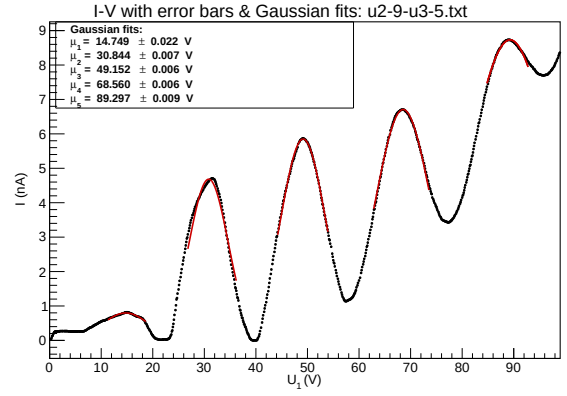


Fig. 9. Plot with error bars and Gaussian fits for the values of $U_2 = 9\text{V}$ and $U_3 = 5\text{V}$.

Energy Differences Between Peaks for Fig.9

Peak Difference	$\Delta E_i \pm \sigma_{\Delta E_i}$
ΔE_1	$16.096 \pm 0.023 \text{ eV}$
ΔE_2	$18.308 \pm 0.009 \text{ eV}$
ΔE_3	$19.408 \pm 0.008 \text{ eV}$
ΔE_4	$20.737 \pm 0.011 \text{ eV}$
Weighted Average	$19.176 \pm 0.005 \text{ eV}$

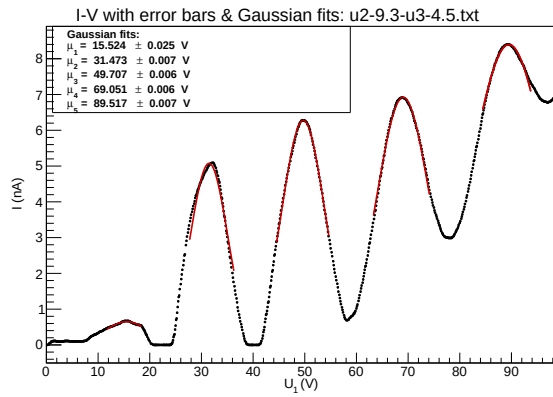


Fig. 8. Plot with error bars and Gaussian fits for the values of $U_2 = 9.3\text{V}$ and $U_3 = 4.5\text{V}$.

Energy Differences Between Peaks for Fig.8

Peak Difference	$\Delta E_i \pm \sigma_{\Delta E_i}$
ΔE_1	$15.949 \pm 0.026 \text{ eV}$
ΔE_2	$18.234 \pm 0.009 \text{ eV}$
ΔE_3	$19.344 \pm 0.008 \text{ eV}$
ΔE_4	$20.466 \pm 0.009 \text{ eV}$
Weighted Average	$19.227 \pm 0.005 \text{ eV}$

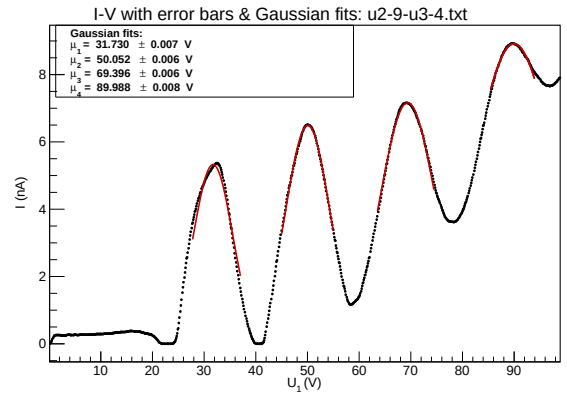


Fig. 10. Plot with error bars and Gaussian fits for the values of $U_2 = 9\text{V}$ and $U_3 = 4\text{V}$.

Energy Differences Between Peaks for Fig.10

Peak Difference	$\Delta E_i \pm \sigma_{\Delta E_i}$
ΔE_1	$18.323 \pm 0.009 \text{ eV}$
ΔE_2	$19.344 \pm 0.008 \text{ eV}$
ΔE_3	$20.592 \pm 0.010 \text{ eV}$
Weighted Average	$19.341 \pm 0.005 \text{ eV}$

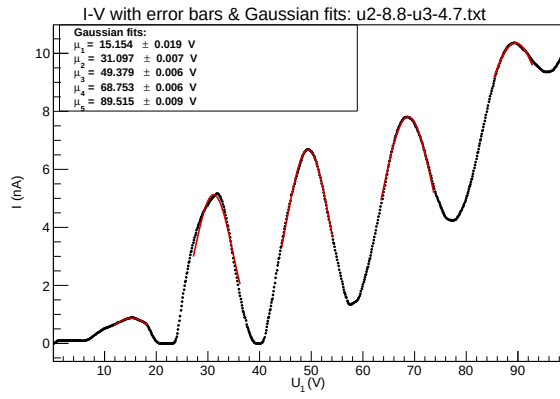


Fig. 11. Plot with error bars and Gaussian fits for the values of $U_2 = 8.8V$ and $U_3 = 4.7V$.

Energy Differences Between Peaks for Fig.11

Peak Difference	$\Delta E_i \pm \sigma_{\Delta E_i}$
ΔE_1	$15.943 \pm 0.020 \text{ eV}$
ΔE_2	$18.282 \pm 0.009 \text{ eV}$
ΔE_3	$19.374 \pm 0.009 \text{ eV}$
ΔE_4	$20.761 \pm 0.011 \text{ eV}$
Weighted Average	$19.075 \pm 0.005 \text{ eV}$

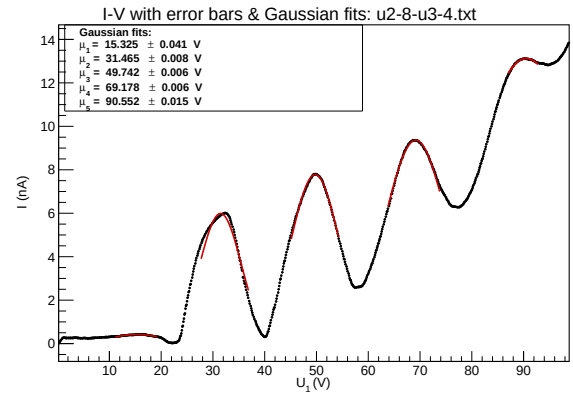


Fig. 13. Plot with error bars and Gaussian fits for the values of $U_2 = 8V$ and $U_3 = 4V$.

Energy Differences Between Peaks for Fig.13

Peak Difference	$\Delta E_i \pm \sigma_{\Delta E_i}$
ΔE_1	$16.140 \pm 0.042 \text{ eV}$
ΔE_2	$18.277 \pm 0.010 \text{ eV}$
ΔE_3	$19.436 \pm 0.009 \text{ eV}$
ΔE_4	$21.374 \pm 0.016 \text{ eV}$
Weighted Average	$19.216 \pm 0.006 \text{ eV}$

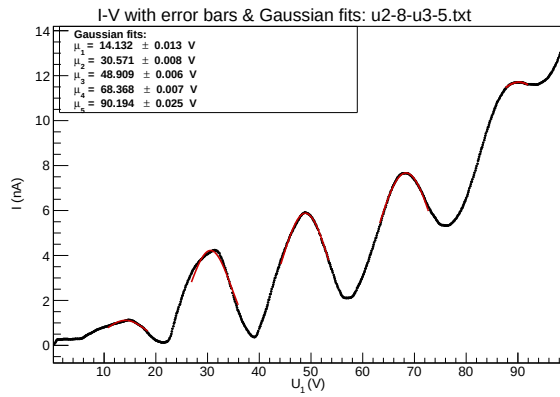


Fig. 12. Plot with error bars and Gaussian fits for the values of $U_2 = 8V$ and $U_3 = 5V$.

Energy Differences Between Peaks for Fig.12

Peak Difference	$\Delta E_i \pm \sigma_{\Delta E_i}$
ΔE_1	$16.439 \pm 0.016 \text{ eV}$
ΔE_2	$18.338 \pm 0.010 \text{ eV}$
ΔE_3	$19.459 \pm 0.010 \text{ eV}$
ΔE_4	$21.826 \pm 0.026 \text{ eV}$
Weighted Average	$18.691 \pm 0.006 \text{ eV}$

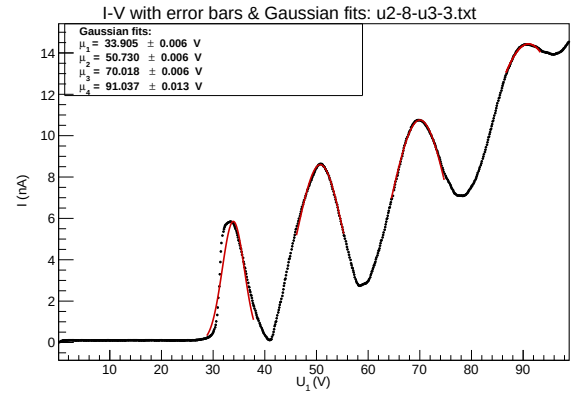


Fig. 14. Plot with error bars and Gaussian fits for the values of $U_2 = 8V$ and $U_3 = 3V$.

Energy Differences Between Peaks for Fig.14

Peak Difference	$\Delta E_i \pm \sigma_{\Delta E_i}$
ΔE_1	$16.826 \pm 0.008 \text{ eV}$
ΔE_2	$19.287 \pm 0.009 \text{ eV}$
ΔE_3	$21.019 \pm 0.014 \text{ eV}$
Weighted Average	$18.392 \pm 0.006 \text{ eV}$

Averaged Excitation Level:

We plotted each data set on a graph with their uncertainties to apply a Gaussian fit on them. We collected the outputs of Gaussian fits and calculated the difference between peaks.

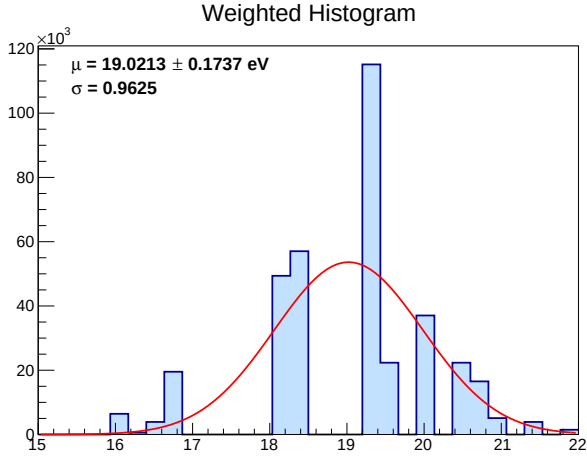


Fig. 16. Excitation level calculated out of 10 data sets. Each weighted average fills the graph as amount of its weight which is $1/\sigma_i^2$. This histogram is filled with directly using the values of peak difference. We have total of 37 different data points in this case.

Finally, we had 10 different weighted averages that are obtained by the peak differences.

In this step, we fill a histogram using these weighted averages. Each average has a weight given by $w_i = 1/\sigma_i^2$. After filling the histogram, we apply a Gauss fit again and obtain a mean value and its error, μ and σ_μ , respectively.

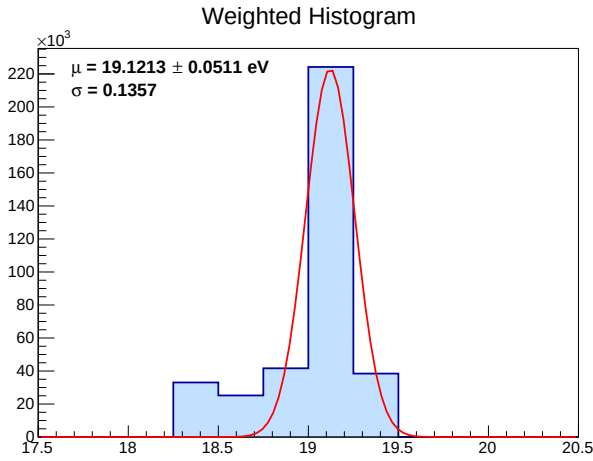


Fig. 15. Excitation level calculated out of 10 data sets. Each weighted average fills the graph as amount of its weight which is $1/\sigma_i^2$. This histogram is filled with averaged values of peak difference. We have total of 10 different data points in this case.

Fig.15 is the first way of doing the final part of the analysis.

Alternatively, we can directly fill histogram with ~ 40 different peak difference values without taking their weighted averages. In this way our data set for the histogram is enlarged, and we expect a more reliable result.

VI. THE RESULT AND THE CONCLUSION

The results we found after 10 experiments under different conditions (i.e. varying U_2 and U_3 values) are

$$\bullet E_{\text{excited}}^{\text{experimental},1} = 19.121 \pm 0.051 \text{ eV}, \quad (13)$$

$$\bullet E_{\text{excited}}^{\text{experimental},2} = 19.021 \pm 0.174 \text{ eV}, \quad (14)$$

These are fairly good values for this kind of experiment. We have too many different kinds of uncertainties in the experimental setup. Some of them may be calibration problems in measurement devices and voltage suppliers. Furthermore, we have a very restricted data set, which is 10 of them. Since in the analysis part, we deal with histograms, data set that is as big as 10 data would be very dangerous to perform the analysis. We might enlarge the amount of data sets taken overall. In addition to those, there are also errors due to the instabilities caused by the changing neon vapor pressure.

However, despite all these facts, we obtained very good results. The acceptable experimental values for the first excitation level (the first after the metastable state) for neon gas are taken as $E_{\text{true}} = 18.7 \text{ eV}$. This value is approximately $\sim 8.2\sigma$ away from the first result and $\sim 1.8\sigma$ away from the second result we found. As expected, the second result is more reliable because its error factor is more realistic for this setup.

REFERENCES

- [1] *Franck-Hertz Experiment Wikipedia*. URL: https://en.wikipedia.org/wiki/Franck_Hertz_experiment.
- [2] Y. Itikawa and N. Mason. “Cross sections for electron collisions with rare gases”. In: *Journal of Physical and Chemical Reference Data* 34.1 (2005), pp. 1–22.
- [3] Sh. et al. Akhmadaliev. “Electron–impact excitation cross sections for neon”. In: *Physical Review A* 103 (2021), p. 012705.
- [4] *Physical Constants*. URL: https://en.wikipedia.org/wiki/Physical_constant.
- [5] G. F. Hanne. “What really happens in the Franck-Hertz experiment with mercury?” In: *Am. J. Phys.* 56 (15 October 1987).
- [6] *Mean Free Path Wikipedia*. URL: https://en.wikipedia.org/wiki/Mean_free_path.
- [7] *Franck-Hertz Experiment*. URL: <https://n.ethz.ch/~marnikm/files/franckHertz.pdf>.
- [8] E. Gülmez. *Advanced Physics Experiments*. 1st. Boğaziçi University Publications, 1999.
- [9] E. Gülmez. *Basic Data Analysis for Experiments in the Physical Sciences*. Boğaziçi University, 1997.

VII. APPENDIX

```
#include <TCanvas.h>
#include <TH1F.h>
#include <TF1.h>
#include <TMath.h>
#include <TRandom3.h>
#include <TLatex.h>
#include <TStyle.h>
#include <algorithm>
```

```

void weighted_histogram_fit() {

    double values[] = {
        16.208, 18.197, 19.266, 20.045,
        18.234, 19.246, 19.980,
        16.817, 18.235, 19.326, 20.121,
        16.826, 19.287, 21.019,
        16.140, 18.277, 19.436, 21.374,
        16.439, 18.338, 19.459, 21.826,
        15.943, 18.282, 19.374, 20.761,
        18.323, 19.344, 20.592,
        16.096, 18.308, 19.408, 20.737,
        15.949, 18.234, 19.344, 20.466
    };
    double uncertainties[] = {
        0.042, 0.009, 0.008, 0.009,
        0.009, 0.008, 0.009,
        0.016, 0.009, 0.009, 0.009,
        0.008, 0.009, 0.014,
        0.042, 0.010, 0.009, 0.016,
        0.016, 0.010, 0.010, 0.026,
        0.020, 0.009, 0.009, 0.011,
        0.009, 0.008, 0.010,
        0.023, 0.009, 0.008, 0.011,
        0.026, 0.009, 0.008, 0.009
    };
    const int N = sizeof(values) /
        sizeof(double);

    TH1F* h = new TH1F("h", "Weighted
        Histogram", 20, 12.0, 35.0);
    h->Sumw2();
    h->SetFillColorAlpha(kAzure + 1, 0.3);
    h->SetLineWidth(2);
    h->SetStats(0);

    for (int i = 0; i < N; i++) {
        double w = 1.0 / (uncertainties[i] *
            uncertainties[i]);
        h->Fill(values[i], w);
    }

    TF1* gausFit = new TF1("gausFit", "gaus",
        15.0, 35.0);
    h->Fit(gausFit, "R");

    double mu = gausFit->GetParameter(1);
    double mu_err = gausFit->GetParError(1);
    double sigma = gausFit->GetParameter(2);

    TCanvas* c1 = new TCanvas("c1",
        "Histogram", 800, 600);
    h->Draw("HIST");
    gausFit->Draw("SAME");

    TLatex text;
    text.SetNDC();
    text.SetTextSize(0.04);
    text.DrawLatex(0.15, 0.85, Form("#mu = %.4f
        #pm %.4f eV", mu, mu_err));
    text.DrawLatex(0.15, 0.80, Form("#sigma =
        %.4f", sigma));

```

```

    c1->Update();
}

// Run: root -l -q
// "process_fh.C(\"data\", \"txt\")"

#include <TSystem.h>
#include <TSystemDirectory.h>
#include <TList.h>
#include <TString.h>
#include <TCanvas.h>
#include <TGraphErrors.h>
#include <TPaveText.h>
#include <TF1.h>
#include <TStyle.h>
#include <TAxis.h>
#include <TH1F.h>

#include <algorithm>
#include <fstream>
#include <sstream>
#include <iostream>
#include <vector>
#include <cmath>
#include <limits>
#include <string>
#include <locale>
#include <iomanip>

struct Peak {
    double target = 0;
    double pickU = 0;
    double pickI = 0;
    double mu = 0;
    double mu_err = 0;
    double A = 0;
    double sigma = 0;
    double chi2ndf = -1;
    bool ok = false;
};

struct LowPoint {
    double U = 0;
    double I = 0;
    bool ok = false;
};

static const double kSigmaX = 0.05;
static const double kSigmaY = 0.01;

static const double kStepV = 17.0;
static const double kSearchHalfWinV = 5.0;
static const double kIminThreshold = 0.4;
static const double kMaxAcceptV = 95.0;
static const double kFitHalfWindowV = 10.0;
static const double kInitSigmaFloor = 0.10;
static const double kSigmaMinLimit = 0.05;
static const double kSigmaLowFactor = 0.4;
static const double kFitWinFromW = 0.6;

static const char kCSV = ',';

static std::string fmt(double x, int prec = 3)

```

```

{
    std::ostringstream oss;
    oss.imbue(std::locale::classic());
    oss << std::fixed <<
        std::setprecision(prec) << x;
    std::string s = oss.str();
    std::replace(s.begin(), s.end(), '.', ',');
    return s;
}

bool readDataset(const std::string& path,
    std::vector<double>& U,
    std::vector<double>& I) {
    U.clear(); I.clear();
    std::ifstream fin(path);
    if (!fin) return false;
    std::string line;
    while (std::getline(fin, line)) {
        if (line.empty()) continue;
        std::stringstream ss(line);
        double u, i;
        if (!(ss >> u >> i)) continue;
        U.push_back(u);
        I.push_back(i);
    }
    return (!U.empty() && U.size() == I.size());
}

std::vector<std::string> listFiles(const char*
    dir, const char* ext) {
    std::vector<std::string> out;
    TSystemDirectory d(dir, dir);
    TList* files = d.GetListOfFiles();
    if (!files) return out;
    TString wanted = "."; wanted += ext;
    TIter it(files);
    while (TObject* obj = it()) {
        auto f = dynamic_cast<TSystemFile*>(obj);
        if (!f || f->IsDirectory()) continue;
        TString name = f->GetName();
        if (name.EndsWith(wanted)) {
            out.emplace_back(TString::Format("%s/%s",
                dir, name.Data()).Data());
        }
    }
    std::sort(out.begin(), out.end());
    return out;
}

void ensureDirs() {
    gSystem->MakeDirectory("plots");
    gSystem->MakeDirectory("results"); }

void drawIV_FrameAndGraph(TCanvas* c,
    TGraphErrors* gr,
    const std::vector<double>& U,
    const std::vector<double>& I,
    const char* title) {
    double xmin = *std::min_element(U.begin(),
        U.end());
    double xmax = *std::max_element(U.begin(),
        U.end());
    double ymin = *std::min_element(I.begin(),
        I.end());
    double ymax = *std::max_element(I.begin(),
        I.end());
    double ypad = 0.06 * std::max(1e-9, ymax -
        ymin);

    ymin -= ypad; ymax += ypad;

    TH1F* frame = c->DrawFrame(xmin, ymin,
        xmax, ymax, title);
    frame->GetXaxis()->SetTitle("U_{1} (V)");
    frame->GetYaxis()->SetTitle("I (nA)");
    frame->GetXaxis()->CenterTitle();
    frame->GetYaxis()->CenterTitle();

    gr->SetMarkerSize(0.0);
    gr->SetLineColor(kBlack);
    gr->SetLineWidth(1);
    gr->Draw("E1 same");
}

void annotatePanelAligned(const
    std::vector<Peak>& peaks) {
    std::vector<Peak> ok;
    for (const auto& p : peaks) if (p.ok)
        ok.push_back(p);
    if (ok.empty()) return;

    double topY = 0.94, botY = std::max(0.70,
        topY - 0.035 * (ok.size() + 1));
    TPaveText* pt = new TPaveText(0.13, botY,
        0.62, topY, "NDC");
    pt->SetFillColorAlpha(0, 0.0);
    pt->SetBorderSize(1);
    pt->SetTextAlign(12);
    pt->SetTextFont(62);
    pt->SetTextSize(0.028);
    pt->AddText("Gaussian fits:");
    for (size_t k = 0; k < ok.size(); ++k) {
        const auto& p = ok[k];
        TString line =
            TString::Format("#mu_{%zu} = %7.3f
                #pm %6.3f V", k + 1, p.mu, p.mu_err);
        pt->AddText(line);
    }
    pt->Draw("same");
}

std::vector<double> buildTargets(double umin,
    double umax) {
    std::vector<double> t;
    double end = std::min(kMaxAcceptV, umax);
    if (end < kStepV) return t;
    int n0 = 1;
    for (int n = n0; ; ++n) {
        double mu0 = n * kStepV;
        if (mu0 > end) break;
        if (mu0 >= umin - kSearchHalfWinV && mu0
            <= umax + kSearchHalfWinV)
            t.push_back(mu0);
    }
    return t;
}

bool pickPeakInWindow(const
    std::vector<double>& U, const
    std::vector<double>& I,
    double center, double halfW,
    double& pickU, double& pickI)
{
    double xmin = center - halfW, xmax = center
        + halfW;
    int best = -1;

```

```

for (size_t i = 0; i < U.size(); ++i) {
    if (U[i] < xmin || U[i] > xmax) continue;
    if (best < 0 || I[i] > I[best]) best =
        (int)i;
}
if (best < 0) return false;
if (I[best] < kIminThreshold) return false;
pickU = U[best];
pickI = I[best];
return true;
}

```

```

struct LowPoint;
LowPoint findMinIInRange(const
    std::vector<double>& U,
    const std::vector<double>& I,
    double Ua, double Ub)
{
    LowPoint lp;
    if (U.empty()) return lp;
    if (Ua > Ub) std::swap(Ua, Ub);

```

```

    int best = -1;
    for (size_t i = 0; i < U.size(); ++i) {
        if (U[i] < Ua || U[i] > Ub) continue;
        if (best < 0 || I[i] < I[best]) best =
            (int)i;
    }
    if (best >= 0) {
        lp.U = U[best];
        lp.I = I[best];
        lp.ok = true;
    }
    return lp;
}

```

```

std::vector<LowPoint>
computeLowsBetweenPeaks(const
    std::vector<double>& U,
    const std::vector<double>& I,
    const std::vector<double>& pickU_sorted)
{
    std::vector<LowPoint> lows;
    const size_t n = pickU_sorted.size();
    if (n == 0) return lows;
    lows.resize(n);

    for (size_t i = 0; i + 1 < n; ++i) {
        double Ua = pickU_sorted[i];
        double Ub = pickU_sorted[i + 1];
        lows[i] = findMinIInRange(U, I, Ua, Ub);
    }
    double Umax = U.empty() ? 0.0 :
        *std::max_element(U.begin(), U.end());
    lows[n - 1] = findMinIInRange(U, I,
        pickU_sorted[n - 1], Umax);
    return lows;
}

```

```

void processOneFile(const std::string& path,
    std::ofstream& csvDeltaE)
{
    std::vector<double> U, I;
    if (!readDataset(path, U, I)) {
        std::cerr << "WARNING: cannot read: " <<
            path << "\n";
    }
}

```

```

return;
}

```

```

const int N = (int)U.size();
auto gr = new TGraphErrors(N);
gr->SetName(("gr_" +
    TString(gSystem->BaseName(path.c_str()))).Data());
for (int i = 0; i < N; ++i) {
    gr->SetPoint(i, U[i], I[i]);
    gr->SetPointError(i, kSigmaX, kSigmaY);
}

```

```

TCanvas* c = new TCanvas(("c_" +
    TString(gSystem->BaseName(path.c_str()))).Data(),
    "", 1000, 700);
gStyle->SetOptStat(0);
c->SetMargin(0.12, 0.04, 0.12, 0.06);

```

```

TString title = TString::Format("I-V with
    error bars & Gaussian fits: %s",
    gSystem->BaseName(path.c_str()));
drawIV_FrameAndGraph(c, gr, U, I, title);

```

```

double umin = *std::min_element(U.begin(),
    U.end());
double umax = *std::max_element(U.begin(),
    U.end());
auto targets = buildTargets(umin, umax);

```

```

std::vector<Peak> picked;
picked.reserve(targets.size());
for (double target : targets) {
    double pU = 0.0, pI = 0.0;
    if (!pickPeakInWindow(U, I, target,
        kSearchHalfWinV, pU, pI)) continue;
    Peak pk; pk.target = target; pk.pickU =
        pU; pk.pickI = pI;
    picked.push_back(pk);
}

```

```

std::vector<double> pickU_sorted;
for (const auto& pk : picked)
    pickU_sorted.push_back(pk.pickU);
std::sort(pickU_sorted.begin(),
    pickU_sorted.end());
auto lows = computeLowsBetweenPeaks(U, I,
    pickU_sorted);

```

```

auto nearestIndex = [&](double u)->int {
    if (pickU_sorted.empty()) return -1;
    auto it =
        std::lower_bound(pickU_sorted.begin(),
            pickU_sorted.end(), u);
    int idx = (it == pickU_sorted.end()) ?
        (int)pickU_sorted.size() - 1
        :
        (int)std::distance(pickU_sorted.begin(),
            it);
    if (idx > 0 && idx <
        (int)pickU_sorted.size())
        if (std::fabs(pickU_sorted[idx] - u)
            > std::fabs(pickU_sorted[idx - 1]
                - u)) idx -= 1;
    return idx;
}

```

```

};

std::vector<Peak> peaks;
    peaks.reserve(picked.size());
for (const auto& pk0 : picked) {
    Peak pk = pk0;

    int idx = nearestIndex(pk.pickU);
    const double INF = 1e12;
    double distLeft = INF;
    double distRight = INF;

    if (idx - 1 >= 0 && idx - 1 <
        (int)lows.size() && lows[idx - 1].ok)
        distLeft = std::fabs(pk.pickU -
            lows[idx - 1].U);
    if (idx >= 0 && idx < (int)lows.size()
        && lows[idx].ok)
        distRight = std::fabs(lows[idx].U -
            pk.pickU);

    double w = std::min(distLeft, distRight);
    if (!(w > 0) || w == INF) w =
        kFitHalfWindowV;
    double sigmaInit =
        std::max(kInitSigmaFloor,
            kSigmaLowFactor * w);
    double fitHalfW = std::max(0.8,
        std::min(kFitHalfWindowV,
            kFitWinFromW * w));

    double xmin = pk.pickU - fitHalfW;
    double xmax = pk.pickU + fitHalfW;

    TF1* fgaus = new
        TF1(Form("fgaus_init_%.1f",
            pk.target), "gaus", xmin, xmax);
    fgaus->SetParNames("A", "mu", "sigma");
    fgaus->SetParameters(pk.pickI, pk.pickU,
        sigmaInit);
    fgaus->SetParLimits(2, kSigmaMinLimit,
        std::max(0.2, w));
    fgaus->SetLineColor(kRed + 1);
    fgaus->SetLineWidth(2);

    int fitStatus = gr->Fit(fgaus, "RQ", "",
        xmin, xmax);
    if (fitStatus == 0) {
        pk.A = fgaus->GetParameter(0);
        pk.mu = fgaus->GetParameter(1);
        pk.sigma = fgaus->GetParameter(2);
        pk.mu_err = fgaus->GetParError(1);
        pk.chi2ndf = (fgaus->GetNDF() > 0) ?
            fgaus->GetChisquare() /
            fgaus->GetNDF() : -1.0;
        pk.ok = true;
        fgaus->Draw("same");
    }
    peaks.push_back(pk);
}

std::vector<Peak> ok;
for (auto& p : peaks) if (p.ok)
    ok.push_back(p);
std::sort(ok.begin(), ok.end(), [](const
    Peak& a, const Peak& b) { return a.mu <
    b.mu; });

```

```

annotatePanelAligned(ok);

ensureDirs();
TString base =
    gSystem->BaseName(path.c_str());
    base.ReplaceAll(".txt", "");
c->Update();
c->SaveAs(TString::Format("plots/%s_peaks.png",
    base.Data()));
c->SaveAs(TString::Format("plots/%s_peaks.pdf",
    base.Data()));

auto nearestIndex2 = [&](double u)->int {
    if (pickU_sorted.empty()) return -1;
    auto it =
        std::lower_bound(pickU_sorted.begin(),
            pickU_sorted.end(), u);
    int idx = (it == pickU_sorted.end()) ?
        (int)pickU_sorted.size() - 1
        :
        (int)std::distance(pickU_sorted.begin(),
            it);
    if (idx > 0 && idx <
        (int)pickU_sorted.size())
        if (std::fabs(pickU_sorted[idx] - u)
            > std::fabs(pickU_sorted[idx - 1]
                - u)) idx -= 1;
    return idx;
};

if (ok.size() >= 2) {
    for (size_t j = 0; j + 1 < ok.size();
        ++j) {
        double dE = ok[j + 1].mu - ok[j].mu;
        double dEerr = std::sqrt(ok[j +
            1].mu_err * ok[j + 1].mu_err +
            ok[j].mu_err * ok[j].mu_err);
        csvDeltaE << base.Data() << kCSV <<
            (j + 1) << kCSV
            << fmt(ok[j].mu, 3) << kCSV <<
                fmt(ok[j].mu_err, 3) << kCSV
            << fmt(ok[j + 1].mu, 3) << kCSV <<
                fmt(ok[j + 1].mu_err, 3) <<
                kCSV
            << fmt(dE, 3) << kCSV <<
                fmt(dEerr, 3) << "\n";
    }
}

csvDeltaE << "\n";

delete c;
}

void process_fh(const char* inputDir = "data",
    const char* ext = "txt") {
    gStyle->SetOptStat(0);
    ensureDirs();

    std::ofstream
        csvDelta("results/deltaE_summary.csv");
    csvDelta << "file" << kCSV << "delta_index"
        << kCSV

```

```

    << "mu_i_V" << kCSV << "sigma_mu_i_V" <<
    kCSV
    << "mu_ip1_V" << kCSV <<
    "sigma_mu_ip1_V" << kCSV
    << "DeltaE_V" << kCSV <<
    "sigma_DeltaE_V\n";

auto files = listFiles(inputDir, ext);
if (files.empty()) {
    std::cerr << "No input files in '" <<
    inputDir << "' with extension '" <<
    ext << "\n";
    return;
}

for (const auto& f : files) {
    std::cout << "Processing: " << f << "\n";
    processOneFile(f, csvDelta);
}

std::cout << "Done. Outputs:\n"
<< " - plots/*.png, *.pdf\n"
<< " - results/peak_fits_summary.csv\n"
<< " - results/deltaE_summary.csv\n";
}

```

and

$$\sigma_\mu^2 \approx \frac{1}{\sum_i \frac{1}{\sigma_i^2}} \quad (20)$$

Our data sets are accessible through the following link. click to access data sets

Error Propagation Formula:

Say

$$y = f(x_1, \dots, x_m) \quad (15)$$

and mean of y becomes

$$\bar{y} = f(\bar{x}_1, \dots, \bar{x}_m) \quad (16)$$

then[9],

$$\sigma_y^2 = \sum_k^m \left(\frac{\partial f}{\partial x_k} \right)^2 \sigma_k^2 + \sum_{k>l}^m \left(\frac{\partial f}{\partial x_k} \right) \left(\frac{\partial f}{\partial x_l} \right) \sigma_{kl}^2 \quad (17)$$

We know how to compute σ , which is roughly $\sigma_y^2 = \frac{1}{N-1} \sum_i^N (f_i - \bar{y})^2$, and if we expand $f(x_1, \dots, x_m)$ around the mean, we get the Eq.(17).

For our case, we have $f = ax \pm by$, so the error will be:

$$\sigma_f^2 = a^2 \sigma_x^2 + b^2 \sigma_y^2 \quad (18)$$

Weighted Average: We have, say, N different mean values and their uncertainties. We want to find a general mean, which is called a mean of means. In this case, we need to consider their weights since they not contribute the mean equally. Since they have different uncertainties, their "closeness to the true value" is not the same. Thus, we should calculate their contribution with respect to their weights, which in many cases are errors.[9]

$$\mu_\mu \approx \frac{\sum_i^N \frac{\mu_i}{\sigma_i^2}}{\sum_i^N \frac{1}{\sigma_i^2}} \quad (19)$$